Memoria de Prácticas

Introducción al Aprendizaje Automático

Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Abril, 2026

Capítulo 1

Cuaderno de Laboratorio — Práctica 2: Selección de Modelos y Regularización

Objetivo

Dominar el protocolo de **Validación Cruzada (K-Fold)** y comprender cómo la regularización **Ridge (L2)** y **Lasso (L1)** previenen el sobreajuste y ayudan a la **selección de atributos**.

1.1. Tarea 1 — Estabilidad de la Validación Cruzada (K-Fold)

Configuración común

Modelo usado: **Regresión Lineal (Ridge con $\lambda \approx 0$)** (regresión lineal sin regularización)
Métrica: **RMSE** (RMSE)
Semillas: **`random_state` 1, 2, 3, 4 y 5, de forma consecutiva para la tarea 1, y el valor 42 para el resto.**

Hipótesis inicial (antes de ejecutar): ¿Crees que el RMSE será estable al cambiar K? No. Con valores muy bajos de K (ej. K=2), el error dependerá en gran medida de qué datos caigan aleatoriamente en entrenamiento y cuáles en test, provocando alta varianza.

1.1.1. Experimento A — K = 2 (Validación simple): variabilidad entre ejecuciones

Ejecuta el modelo **varias veces** con K=2 (por ejemplo 5–10) y registra el RMSE.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.33 | Time=0.0046s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.43 | Time=0.0050s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.24 | Time=0.0055s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.17 | Time=0.0046s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.54 | Time=0.0057s
```

Registro de resultados (rellena):

Ejecución	RMSE
1	5.33
2	5.43
3	5.24
4	5.17
5	5.54

Análisis:

- ¿Por qué el error varía tanto entre ejecuciones? Porque el conjunto de entrenamiento es pequeño (50 % de los datos). El modelo es extremadamente sensible a la partición aleatoria específica de cada ejecución.
- ¿Es fiable esta métrica (con K=2) para tomar una decisión técnica? No, su alta varianza impide saber si una mejora aparente se debe a la configuración del modelo o simplemente a la aleatoriedad de la división de datos.

1.1.2. Experimento B — K = 10 (estándar): RMSE más estable

Ejecuta el modelo con K=10 y anota el RMSE.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.21 | Time=0.0190s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.19 | Time=0.0135s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.16 | Time=0.0154s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.17 | Time=0.0179s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.23 | Time=0.0161s
```

RMSE observado (K=10): **5.19 (Media aprox.)**

Análisis: Describe si el resultado parece más estable que en K=2 y por qué. Sí, es notablemente más estable (fluctúa entre 5.16 y 5.23). Al usar K=10, el modelo entrena con el 90 % de los datos en cada iteración y promedia 10 errores diferentes, mitigando drásticamente el impacto de particiones anómalas.

1.1.3. Experimento C — K = 100 (Leave-One-Out): coste vs. precisión

Ejecuta el modelo con K=100 (LOOCV) y observa el RMSE y el tiempo.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1252s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1395s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.17 | Time=0.1390s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1290s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.19 | Time=0.1231s
```

RMSE observado (K=100): **5.18** Tiempo aproximado: **0.131s**

Análisis: Compara el coste computacional frente a K=10. El tiempo de ejecución se multiplica casi por 8 (de $\sim$ 0,016s a $\sim$ 0,131s). ¿Merece la pena el tiempo extra para mejorar la precisión obtenida? No merece la pena. La mejora en la precisión del RMSE es marginal (apenas 0.01 unidades), lo cual no justifica el enorme aumento del coste computacional.

1.1.4. Resumen comparativo (K = 2 vs 10 vs 100)

K	RMSE	Comentario de estabilidad/coste
2	5.17 - 5.54	Alta inestabilidad (varianza). Coste muy bajo.
10	5.16 - 5.23	Alta estabilidad. Coste equilibrado y eficiente.
100	5.17 - 5.19	Máxima estabilidad. Coste computacional prohibitivo.

Conclusión Tarea 1: ¿Cuál elegirías para este problema y por qué? Elegiría K=10. Demuestra ser el estándar óptimo porque ofrece un error robusto y fiable para comparar modelos, sin incurrir en los altos tiempos de ejecución que exige K=100. Proporciona el mejor compromiso técnico entre precisión estadística y eficiencia temporal.

1.2. Tarea 2 — Regularización Ridge vs. Lasso (K = 10)

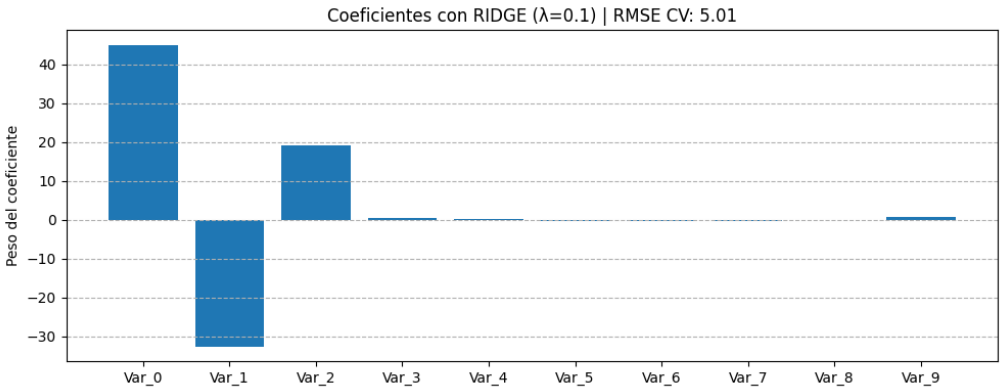
Configuración

Fijamos K = 10 y comparamos:

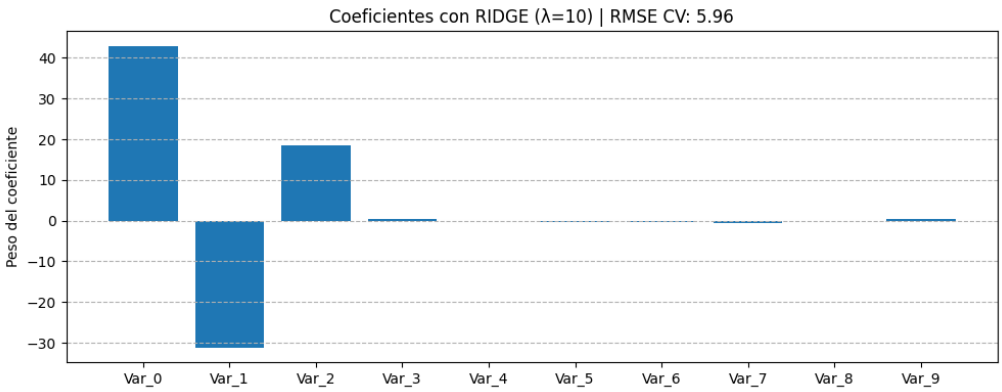
- **Ridge Regression** (penalización L2)
- **Lasso Regression** (penalización L1)

1.2.1. Ridge (L2) — $\lambda = 0{,}1,\ 10,\ 100$

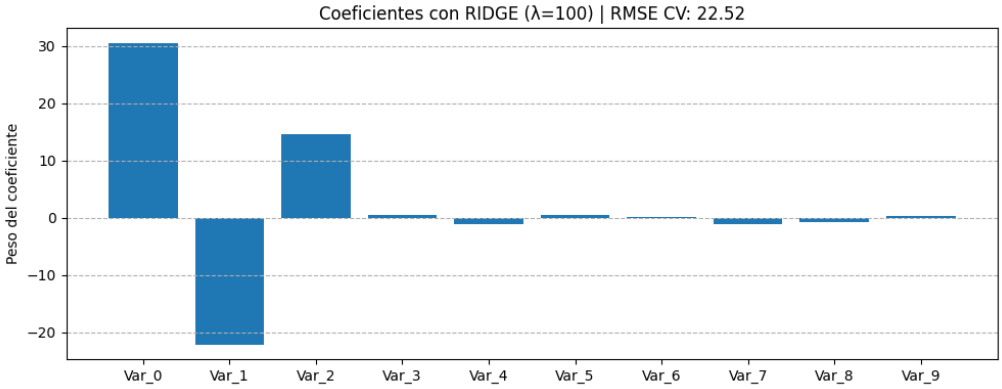
Resultados: captura el gráfico de barras de coeficientes para cada λ.



RMSE Ridge ($\lambda = 0{,}1$): **5.01**



RMSE Ridge ($\lambda = 10$): **5.96**

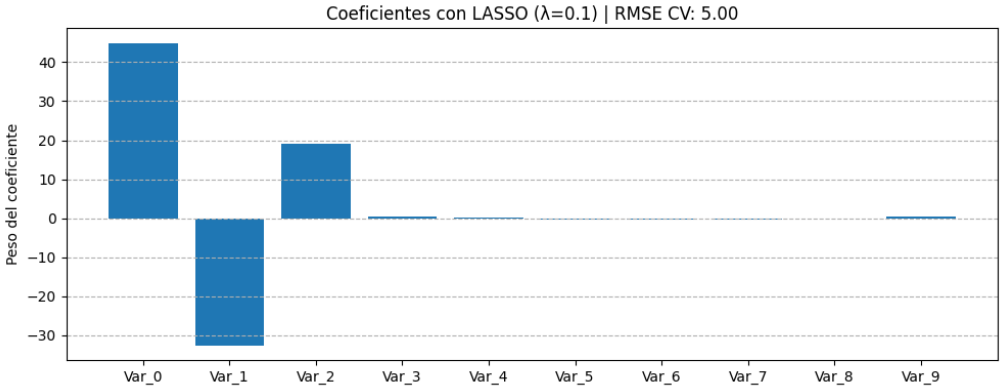


RMSE Ridge ($\lambda = 100$): **22.52**

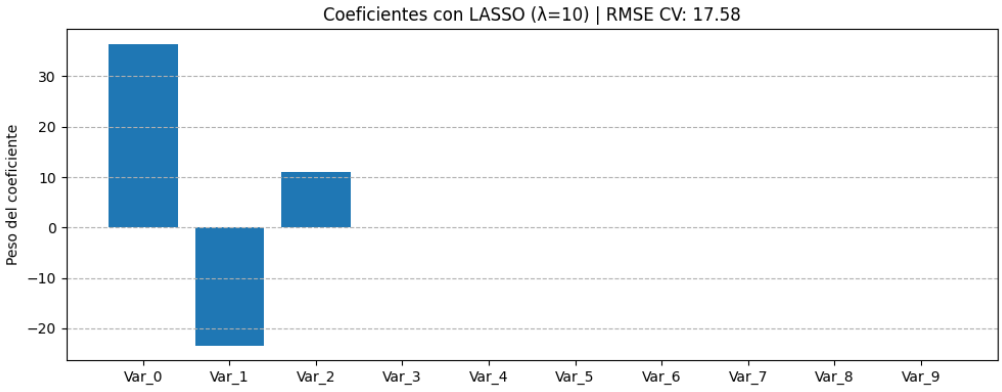
Análisis: ¿Llegan a valer **cero** los coeficientes de las variables ruidosas?. No llegan a valer cero absoluto. Visualmente en las gráficas parece que algunas como (`Var_8` desaparecen, pero técnicamente Ridge (L2) simplemente reduce o .^{en}coge"drásticamente la magnitud de los coeficientes penalizados, distribuyendo el peso entre todos sin llegar a anularlos por completo.

1.2.2. Lasso (L1) — $\lambda = 0{,}1,\ 10,\ 100$

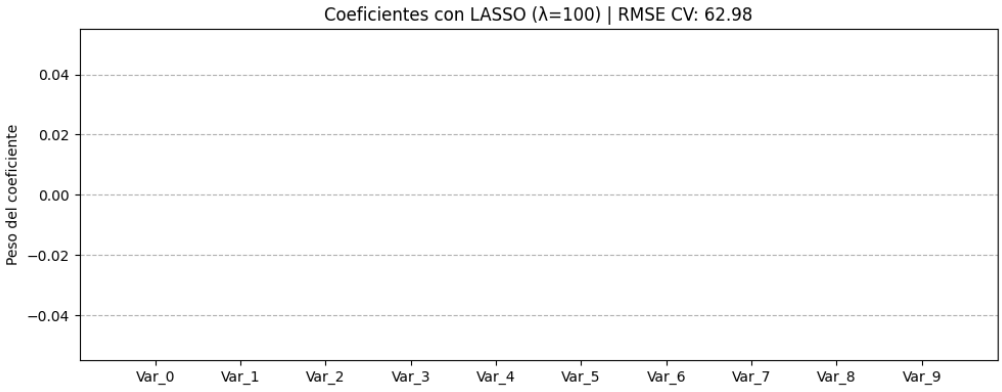
Resultados: captura el gráfico de barras de coeficientes para cada λ.



RMSE Lasso ($\lambda = 0{,}1$): **5.00**



RMSE Lasso ($\lambda = 10$): **17.58**



RMSE Lasso ($\lambda = 100$): **62.98**

Análisis clave:

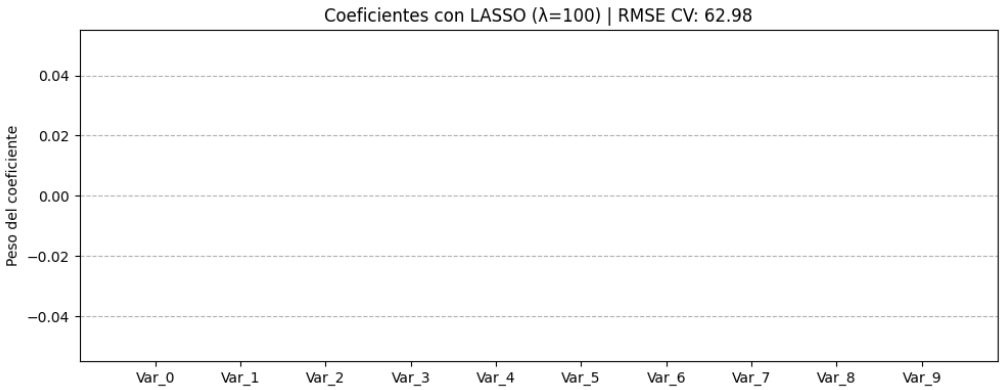
1. Identifica para qué valor de λ el modelo **Lasso** consigue **eliminar** (poner a cero) las 7 variables ruidosas, quedándose solo con las 3 importantes:

$$\lambda^* = \mathbf{1.0} (\text{y en general entre 0.5 y 2.0})$$

2. Variables con coeficiente distinto de cero (modelo Lasso con λ^*):
Var_0, Var_1, Var_2

1.2.3. El dilema de la complejidad — λ demasiado alto

Prueba un valor extremo, por ejemplo $\lambda = 100$, y registra qué ocurre.



RMSE ($\lambda = 100$): **62.98**

Análisis: Explica qué ocurre con el RMSE si aplicas un λ demasiado alto. El RMSE se dispara exageradamente (hasta 62.98). ¿Estamos ante un caso de **sesgo alto** o **varianza alta**? Justifica. Estamos ante un caso evidente de **sesgo alto (underfitting)**. La penalización es tan severa que el modelo se vuelve excesivamente rígido, empujando todos los coeficientes hacia cero e impidiendo que pueda capturar la señal o patrón real de los datos.

1.3. Reto — El "Cazador de Variables"

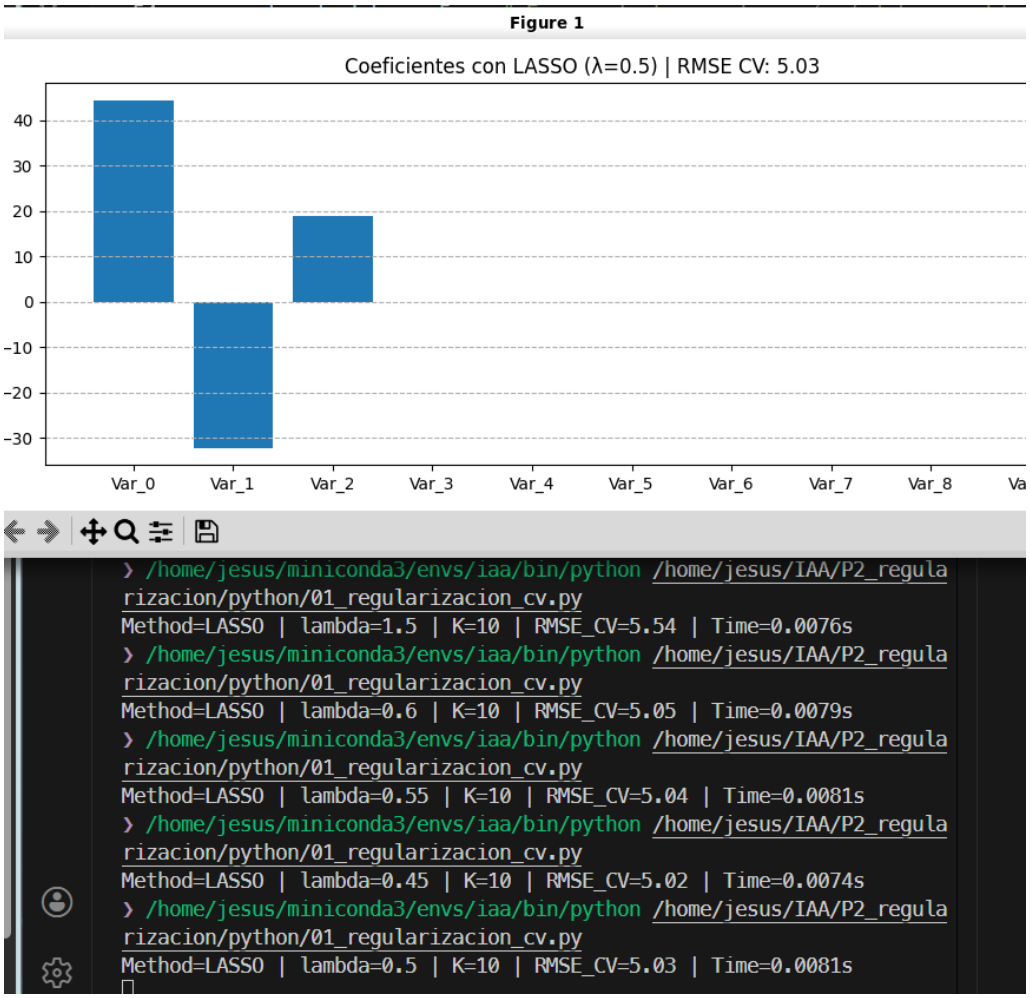
Contexto: Imagina que cada variable incluida en tu modelo tiene un **coste de mantenimiento**. Tu objetivo es encontrar el modelo **más sencillo posible** (menos variables con coeficientes $\neq 0$) que mantenga un RMSE inferior a un umbral:

$$\text{RMSE} < \mathbf{5.50} \quad \text{(umbral asumido)}$$

Mejor solución encontrada (rellena):

$$\lambda = \mathbf{0.5} \qquad \#\text{Variables activas} = \mathbf{3} \qquad \text{RMSE} = \mathbf{5.03}$$

Variables seleccionadas (coeficiente $\neq 0$): **Var_0, Var_1 y Var_2**



Justificación técnica (6–10 líneas): ¿Por qué este modelo es el mejor compromiso entre simplicidad (coste) y rendimiento (RMSE)? Tras realizar un ajuste paramétrico fino, $\lambda = 0{,}5$ demuestra ser el punto de equilibrio óptimo. Logra la máxima simplicidad técnica al descartar completamente los 7 atributos que solo aportaban ruido, minimizando así los futuros costes de mantenimiento e ingesta de datos del modelo. Simultáneamente, conserva un rendimiento excelente con un RMSE de 5.03, el cual es prácticamente idéntico al de modelos menos restrictivos (como $\lambda = 0{,}1$ con RMSE 5.00). Nos quedamos con la señal real y descartamos el ruido sin penalizar el error.

1.4. Conclusiones finales

Resume en 8–12 líneas lo aprendido sobre:

- Estabilidad de K-Fold al variar K (K=2 vs K=10 vs LOOCV).
- Diferencias prácticas entre Ridge y Lasso (coeficientes, selección de variables).
- Relación entre λ grande y el sesgo/varianza.

La validación cruzada con $K = 10$ es el estándar porque equilibra un RMSE estadísticamente fiable frente a la inestabilidad de $K = 2$, evitando los desproporcionados tiempos de ejecución de LOOCV ($K = 100$). En cuanto a la regularización, Ridge (L2) reduce la magnitud de los coeficientes de las variables menos relevantes pero los mantiene activos, mientras que Lasso (L1) es capaz de forzarlos a cero absoluto. Esta propiedad hace que Lasso actúe como un potente selector automático de características, ideal para limpiar datasets con ruido. Por último, un hiperparámetro λ demasiado alto penaliza en exceso al modelo, volviéndolo incapaz de aprender (underfitting) y empujándolo a una zona de sesgo alto donde el error se dispara.